



UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL CALCULATOARE

Sistem de Control FPGA pentru Mână Robotică cu Filtrare Kalman

LUCRARE DE LICENȚĂ

Absolvent: **Radu-Rareș CIOBANU**

Coordonator **Prof. dr. ing. Radu Gabriel DĂNESCU**
științific:

2026



UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL CALCULATOARE

DECAN,
Prof. dr. ing. Vlad MUREȘAN

DIRECTOR DEPARTAMENT,
Prof. dr. ing. Rodica POTOLEA

Absolvent: **Radu-Rareș CIOBANU**

Sistem de Control FPGA pentru Mână Robotică cu Filtrare Kalman

1. **Enunțul temei:** *Proiectarea și implementarea unui sistem de control pe FPGA pentru o mână robotică cu 8 servomotoare, utilizând filtrare Kalman pentru stabilizarea mișcării și feedback de la un ADC pe 24 de biți.*
2. **Conținutul lucrării:** *Introducere, Obiectivele proiectului, Studiu bibliografic, Analiză și fundamentare teoretică, Proiectare de detaliu și implementare, Testare și validare, Manual de instalare și utilizare, Concluzii, Bibliografie, Anexe.*
3. **Locul documentării:** Universitatea Tehnică din Cluj-Napoca, Departamentul Calculatoare
4. **Consultanți:**
5. **Data emiterii temei:** 1 Octombrie 2025
6. **Data predării:** 10 Iulie 2026

Absolvent: _____

Coordonator științific: _____

**UNIVERSITATEA TEHNICĂ**
DIN CLUJ-NAPOCA**FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE**
DEPARTAMENTUL CALCULATOARE**Declarație pe propria răspundere privind
autenticitatea lucrării de licență**

Subsemnatul(a) _____, _____ legitimat(ă) cu

_____ seria _____ nr. _____
(conform copiei anexate prezentei declarații, copie sem-
nată și certificată “conform cu originalul”), autorul lucrării

elaborată în vederea susținerii examenului de finalizare a studiilor de
licență la Facultatea de Automatică și Calculatoare, Programul de studiu
_____ din cadrul Universității Tehnice
din Cluj-Napoca, sesiunea _____ a anului universitar _____,
declar pe proprie răspundere, că această lucrare este rezultatul propriei activități
intelectuale, pe baza cercetărilor mele și pe baza informațiilor obținute din surse care au
fost citate, în textul lucrării și în bibliografie.

Declar, că această lucrare nu conține porțiuni plagiate, iar sursele bibliografice au
fost folosite cu respectarea legislației române și a convențiilor internaționale privind
drepturile de autor.

Declar, de asemenea, că această lucrare nu a mai fost prezentată în fața unei alte comisii
de examen de licență.

În cazul constatării ulterioare a unor declarații false, voi suporta sancțiunile administra-
tive, respectiv, *anularea examenului de licență*.

Data

Nume, Prenume

Semnătura

Cuprins

Capitolul 1	Proiectare de detaliu și implementare	1
1.1	Prezentarea generală a sistemului	1
1.1.1	Descrierea soluției implementate	1
1.1.2	Diagrama de sistem	3
1.1.3	Mediul de dezvoltare și platforma	3
1.2	Realizarea fizică a mâinii robotice	4
1.2.1	Combinarea modelelor și adaptările aduse	4
1.2.2	Integrarea servomotoarelor și sistemul de pulley-uri	4
1.2.3	Sistemul de acționare prin fire și arcuri	5
1.2.4	Sistemul de feedback al servomotoarelor	6
1.2.5	Fabricarea prin printare 3D	7
1.3	Realizarea montajului electronic	9
1.3.1	Izolarea și translatarea de nivel a semnalelor PWM	9
1.3.2	Bucula de feedback și normalizarea semnalelor	10
1.3.3	Alimentarea montajului	10
1.4	Implementarea pe placa FPGA	11
1.4.1	Arhitectura modulară	12
1.4.2	Generarea semnalului de sincronizare	12
1.4.3	Recepția și validarea cadrelor	12
1.4.4	Sincronizarea prin mailbox	13
1.4.5	Implementarea filtrului Kalman	13
1.4.6	Împărțitorul în virgulă fixă	17
1.4.7	Generarea semnalelor PWM	17
1.4.8	Interfața cu convertorul analog-digital	18
1.4.9	Modurile de funcționare	18
1.5	Închiderea temporală a circuitului	18
1.5.1	Problema inițială	19
1.5.2	Soluțiile aplicate	19
1.5.3	Rezultatele obținute	19
1.5.4	Utilizarea resurselor	20
	Bibliografie	21

Capitolul 1. Proiectare de detaliu și implementare

Dacă în capitolul precedent au fost prezentate fundamentele teoretice ale sistemului, capitolul de față arată modul concret în care acestea au fost transpuse în realitate. Sunt descrise, pe rând, realizarea fizică a mâinii robotice, de la combinarea modelelor printate 3D până la montarea sistemului de acționare și a celui de feedback, precum și implementarea pe placa FPGA, cu detalierea fiecărui modul și a modului în care acestea conlucrează. O atenție deosebită este acordată închiderii temporale a circuitului, una dintre cele mai dificile etape ale proiectării digitale.

1.1. Prezentarea generală a sistemului

1.1.1. Descrierea soluției implementate

Valorificând fundamentele teoretice prezentate în capitolul precedent, sistemul dezvoltat în cadrul acestei lucrări realizează controlul unei mâini robotice printate 3D, ale cărei articulații urmăresc în timp real mișcările mâinii unui operator uman. Soluția se împarte în mod natural în două componente strâns legate între ele. Pe de o parte se află ansamblul mecanic, alcătuit din piese printate 3D, fire de acționare, arcuri și servomotoare, care reproduce structura și mișcările unei mâini umane, pe de altă parte se află partea electronică și digitală, implementată pe placa FPGA, care primește comenzile, le prelucerează și le transformă în mișcarea efectivă a articulațiilor.

Elementul central al implementării digitale este filtrul Kalman, care prelucerează unghiurile primite înainte ca acestea să fie transformate în comenzi pentru servomotoare. Necesitatea acestui filtru vine din faptul că unghiurile provenite din sistemul de viziune, deși corecte în ansamblu, conțin variații bruște și zgomot provenit din procesul de detecție vizuală. Trimise direct către servomotoare, aceste variații ar produce mișcări sacadate și tremurânde, nepotrivite cu gestul intenționat și solicitante pentru mecanism. Filtrul tratează fiecare dintre cele opt articulații ca pe un canal independent, modelat printr-un sistem cu două stări, poziție și viteză, și produce pentru fiecare o traiectorie netedă, care urmărește fidel mișcarea reală, eliminând în același timp tremurul. Un detaliu important al implementării este că toate cele opt canale sunt prelucrate de aceeași unitate de calcul, în mod secvențial, în interiorul unui singur ciclu de control, soluție care economisește resurse fără a compromite respectarea constrângerilor de timp.

Pentru a putea funcționa, partea digitală are nevoie ca unghiurile să ajungă la ea într-o formă bine definită și verificată. Acest rol revine modului de recepție, care extrage unghiurile dintr-un cadru transmis prin UART și le validează cu ajutorul unei sume de control, respingând cadrele afectate de erori de transmisie. Întrucât cadrele sosesc la momente dictate de calculatorul gazdă, fără legătură cu ritmul intern al plăcii, între recepție și filtru a fost adăugat un mecanism de sincronizare de tip cutie poștală, care garantează că filtrul lucrează întotdeauna pe un set de date stabil și complet. Pornind de la unghiurile filtrate, un generator dedicat produce cele opt semnale PWM care comandă efectiv poziția servomotoarelor, fiecare canal dispunând de propriul generator, astfel încât toate semnalele sunt produse simultan.

Pe lângă funcția principală de urmărire a mâinii, sistemul oferă și un mod de

funcționare autonomă. Acesta poate fi activat de pe placă și determină mâna să execute o secvență predefinită de mișcări, stocată într-o memorie internă, fără a mai fi necesară conexiunea cu calculatorul gazdă. Modul autonom este util atât pentru demonstrarea funcționării, cât și pentru testarea mecanică a ansamblului, independent de partea de viziune. În paralel cu toate acestea funcționează și o buclă de monitorizare, prin care un convertor analog-digital citește poziția reală a servomotoarelor și o transmite înapoi către placă, permițând observarea modului în care articulațiile răspund la comenzi.

Pentru o mai bună înțelegere a modului în care aceste componente conlucrează, este utilă urmărirea drumului parcurs de o singură comandă, de la mișcarea operatorului până la mișcarea mâinii robotice. Sistemul de viziune detectează poziția mâinii și calculează cele opt unghiuri corespunzătoare articulațiilor. Aceste unghiuri sunt împachetate într-un cadru și transmise prin UART către placă. Modulul de recepție preia cadrul, verifică suma de control și depune unghiurile în cutia poștală. La începutul fiecărui ciclu de control, care se repetă o dată la 20 de milisecunde, filtrul Kalman preia cel mai recent set de unghiuri disponibil și îl prelucrează, canal cu canal, producând unghiurile netezite. Acestea sunt transmise generatoarelor PWM, care le transformă în impulsuri de durată corespunzătoare, iar impulsurile, după trecerea prin izolatoarele galvanice, ajung la servomotoare și produc mișcarea articulațiilor. Între timp, convertorul analog-digital citește pozițiile reale și le trimite înapoi pe placă, închizând astfel ciclul de observare.

În cele ce urmează, cele două componente sunt tratate separat. Mai întâi este descrisă realizarea fizică a mâinii, cu accent pe provocările întâmpinate la combinarea modelelor și la montarea sistemului de acționare, după care se trece la implementarea digitală pe placa FPGA, prezentată modul cu modul.

1.1.2. Diagrama de sistem

Privit în ansamblu, sistemul este alcătuit din lanțul de componente reprezentat în Figura 1.1.

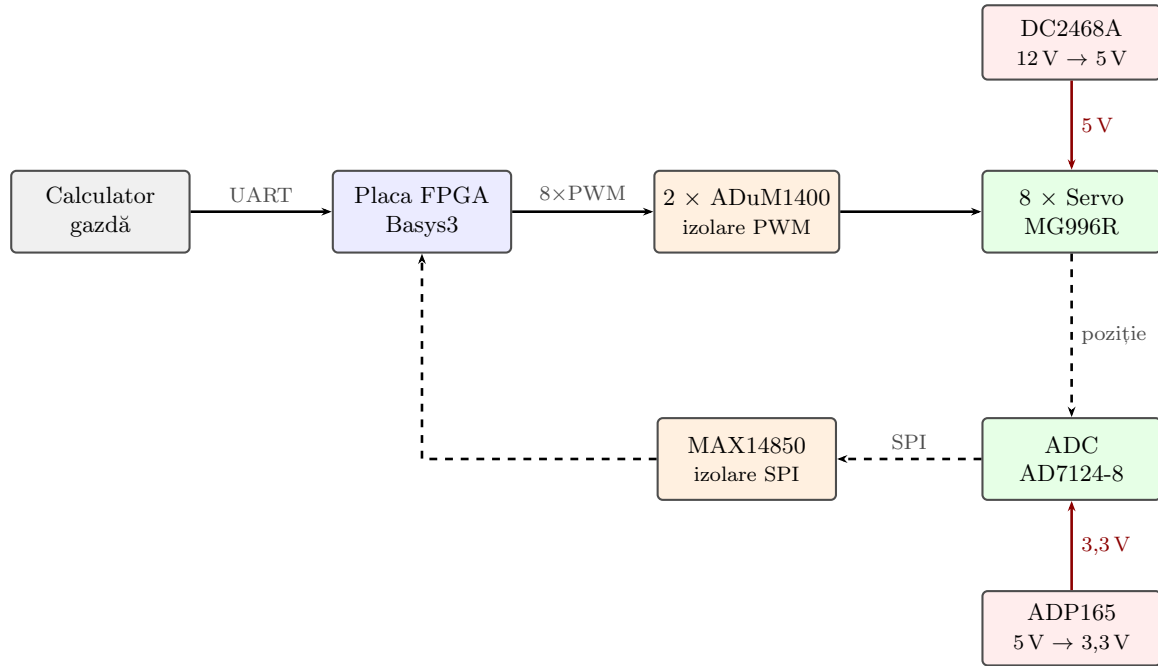


Figura 1.1: Diagrama de sistem, cu lanțul principal de comandă, bucla de monitorizare prin convertorul analog-digital și sursele de alimentare.

La un capăt se află calculatorul gazdă, care transmite unghiurile prin UART, iar la celălalt capăt se găsesc cele opt servomotoare care produc mișcarea. Între ele, placa Basys3 prelucrează datele și comandă servomotoarele prin intermediul a două izolatoare ADuM1400, care asigură separarea galvanică a celor opt semnale PWM. Alimentarea servomotoarelor este furnizată de modulul DC2468A, care coboară tensiunea la 5 V.

Bucloa de monitorizare, reprezentată prin linii întrerupte, pornește de la servomotoare către convertorul AD7124-8, care citește poziția reală, și se întoarce la placă prin izolatorul SPI MAX14850. Convertorul este alimentat la 3,3 V printr-un regulator liniar ADP165, separat de alimentarea servomotoarelor pentru a-i asigura o tensiune curată. Această organizare reflectă fidel principiile de izolare și de alimentare fundamentate în capitolul precedent.

1.1.3. Mediul de dezvoltare și platforma

Partea digitală a fost descrisă în limbajul VHDL și dezvoltată în mediul Vivado, folosit atât pentru sinteză și implementare, cât și pentru analiza temporală a circuitului. Platforma de execuție este placa Basys3, construită în jurul unui circuit FPGA Xilinx Artix-7 XC7A35T, care funcționează la o frecvență de ceas de 100 MHz. Placa pune la dispoziție conectoare Pmod prin care sunt scoase semnalele PWM către servomotoare și magistrala SPI către convertor, precum și o interfață UART prin portul USB.

Partea mecanică a fost modelată și pregătită pentru printare în mediul Bambu Studio, iar piesele au fost fabricate pe o imprimantă Bambu Lab A1. Detaliile privind

parametrii de printare și adaptările aduse modelelor sunt prezentate în secțiunea dedicată realizării fizice a mâinii.

1.2. Realizarea fizică a mâinii robotice

Construcția fizică a mâinii a reprezentat o etapă amplă și plină de provocări, în care modelele open-source studiate în capitolul de studiu bibliografic au fost combinate, adaptate și completate cu piese proprii, până la obținerea unui ansamblu funcțional. Această secțiune descrie pe rând combinarea modelelor, integrarea servomotoarelor, sistemul de acționare prin fire și arcuri, sistemul de feedback și, în final, fabricarea pieselor prin printare 3D.

1.2.1. Combinarea modelelor și adaptările aduse

Punctul de plecare l-a constituit studiul modelelor existente și încercarea de a le combina într-un singur ansamblu coerent. Fiecare model a contribuit cu partea sa cea mai potrivită. De la proiectul InMoov a fost preluat mecanismul de rotație a încheieturii, apreciat pentru modul bine gândit în care ghidează firele, de la modelul dezvoltat de Ryonicle au fost preluate palma și degetele, iar pentru zona de la încheietură în jos, mai exact antebrațul și articulația cotului, s-a pornit de la modelul propus de Hussein. Combinarea acestor componente provenite de la autori diferiți nu a fost însă directă, deoarece ele nu fuseseră gândite pentru a fi îmbinate între ele, având dimensiuni și interfețe de prindere diferite.

Din acest motiv a fost necesară o muncă susținută de modelare și remodelare, prin care piesele să fie aduse la o scară comună și să poată fi cuplate corect. Cea mai importantă intervenție a vizat îmbinarea dintre antebraț și încheietură, zonă în care cele două modele nu se potriveau. Pentru a rezolva această problemă a fost modelată o piesă proprie, sub forma unui semicerc prevăzut cu găuri, prezentată în Figura 1.2, care face legătura între partea superioară și partea inferioară a antebrațului, asigurând o prindere solidă și aliniată a celor două subansamble. Această piesă, alături de celelalte adaptări, a transformat colecția de modele separate într-un braț unitar.

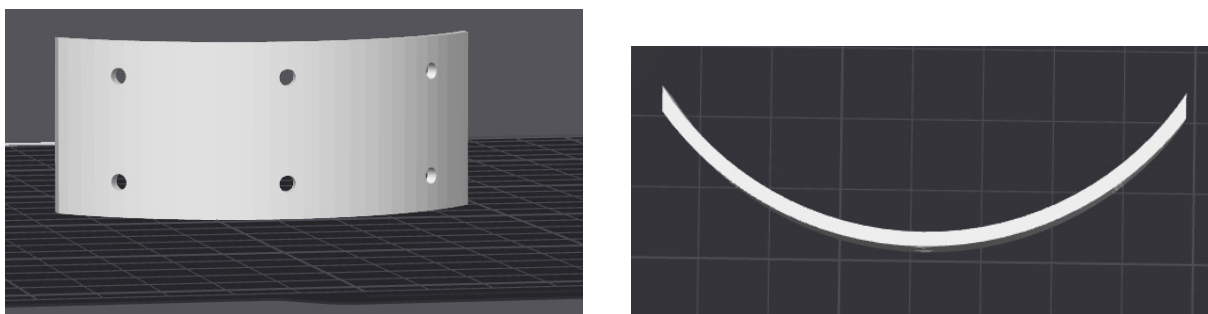


Figura 1.2: Semicercul propriu de îmbinare a antebrațului, vedere laterală și de sus.

1.2.2. Integrarea servomotoarelor și sistemul de pulley-uri

Una dintre cele mai dificile provocări mecanice a fost amplasarea celor opt servomotoare în interiorul ansamblului. Spațiul disponibil este limitat, iar servomotoarele alese au o cursă de 180 de grade, ceea ce a impus o gândire atentă a modului de prindere și de transmitere a mișcării. Un servomotor cu o cursă mai mare, de exemplu de 270 de grade, ar fi oferit mai multă libertate, însă cuplul ridicat și disponibilitatea servomotoarelor

folosite au cântărit mai mult în alegerea finală.

Cea mai delicată parte a fost transmiterea mișcării prin sistemul de pulley-uri. Servomotoarele care acționează degetele sunt dispuse pe două rânduri. Pe rândul din spate se află două servomotoare, unul cu pulley înălțat și unul cu pulley obișnuit, iar pe rândul din față se află trei, dintre care cele de pe margini, din stânga și din dreapta, au pulley înălțat, iar cel din mijloc are pulley obișnuit. Pulley-urile înălțate au fost necesare pentru a ridica șanțul firului față de planul servomotorului, astfel încât firele și arcurile care vin de la articulațiile mai îndepărtate să treacă pe deasupra servomotoarelor vecine, fără a se freca de ele și fără a se bloca. Diferența dintre cele două variante este vizibilă în Figura 1.3.

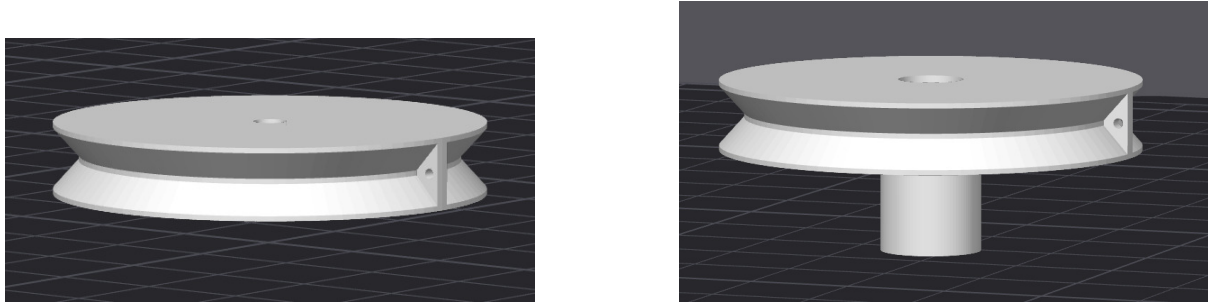


Figura 1.3: Pulley obișnuit (stânga) și pulley înălțat (dreapta).

1.2.3. Sistemul de acționare prin fire și arcuri

Acționarea degetelor se realizează prin fire care joacă rolul tendoanelor, conform principiului prezentat în capitolul de analiză. Pentru aceste fire a fost ales un fir de pescuit din comerț, foarte fin, cu un diametru de 0,27 mm și o rezistență la rupere de 14 kg. Finețea firului îi permite să alunece prin canalele de ghidare din interiorul pieselor, iar rezistența ridicată asigură că suportă fără probleme forțele dezvoltate de servomotoare.

Firul nu leagă însă direct servomotorul de deget, ci este împărțit în două segmente, între care se află un arc. Astfel, un segment merge de la deget la arc, iar celălalt de la arc la servomotor. Arcurile folosite au fost procurate dintr-un târg de vechituri și au toate aceeași rezistență, astfel încât preiau tensiunea în mod uniform pe toate canalele. Rolul lor, detaliat în capitolul de analiză, este de a prelua vârfurile de tensiune și de a proteja firul și mecanismul în cazul unor solicitări bruște. Revenirea degetelor în poziția deschisă se face pasiv, cu ajutorul unor elemente elastice care joacă rolul tendoanelor extensoare. Figura 1.4 prezintă arcurile montate în serie pe fire, în interiorul antebrățului, pe traseul dintre servomotoare și degete.



Figura 1.4: Arcurile montate în serie pe firele de acționare.

1.2.4. Sistemul de feedback al servomotoarelor

Una dintre cele mai laborioase etape ale construcției a fost realizarea sistemului de feedback, prin care poziția reală a fiecărui servomotor poate fi citită de placa FPGA. Servomotoarele folosite nu oferă în mod nativ un semnal de poziție la exterior, astfel încât acesta a trebuit obținut prin intervenție directă. Fiecare servomotor a fost desfăcut, iar pe pinul central al potențiometrului intern, care indică poziția axului, a fost lipit un fir suplimentar, așa cum se vede în Figura 1.5. În acest fel, tensiunea de pe potențiomtru, proporțională cu unghiul servomotorului, devine accesibilă în exterior.



Figura 1.5: Firul de feedback lipit pe potențiometrul intern al servomotorului.

Toate aceste fire de feedback au fost organizate în funcție de canal, prinse cu cleme,

etichetate și extinse până în zona umărului brațului, unde au fost grupate într-un singur loc, alături de firele de comandă, pentru a fi mai ușor de gestionat. Figura 1.6 arată acest fascicul de fire, fiecare purtând o etichetă cu articulația căreia îi corespunde.

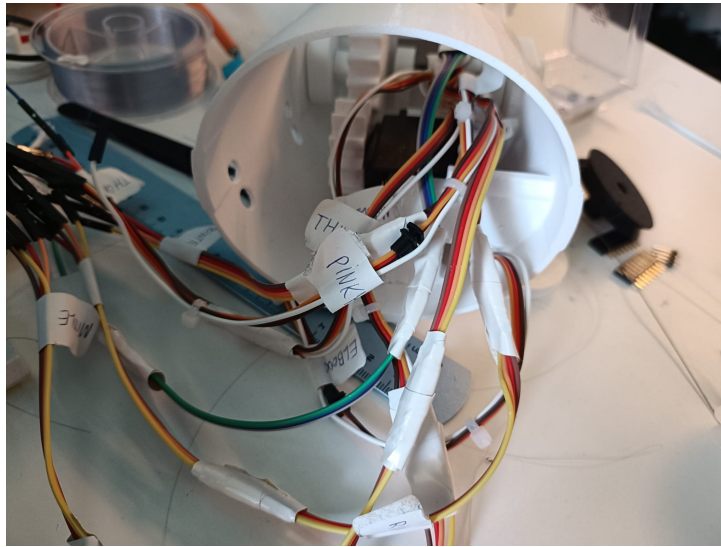


Figura 1.6: Fasciculul de fire de feedback, etichetate pe articulații.

Un aspect important este că semnalul de feedback nu este uniform de la un servomotor la altul. La poziția de 0 grade, tensiunea măsurată este de aproximativ 0,382 V, iar la 180 de grade ajunge la circa 2,9 V. Pentru ca aceste valori să poată fi preluate corect de convertor, fără a depăși tensiunea sa de referință de 2,5 V, fiecare semnal de feedback este atenuat printr-un divizor rezistiv, montat pe o plăcuță separată, prezentată în Figura 1.7. Detaliile divizorului sunt prezentate în secțiunea dedicată montajului electronic.

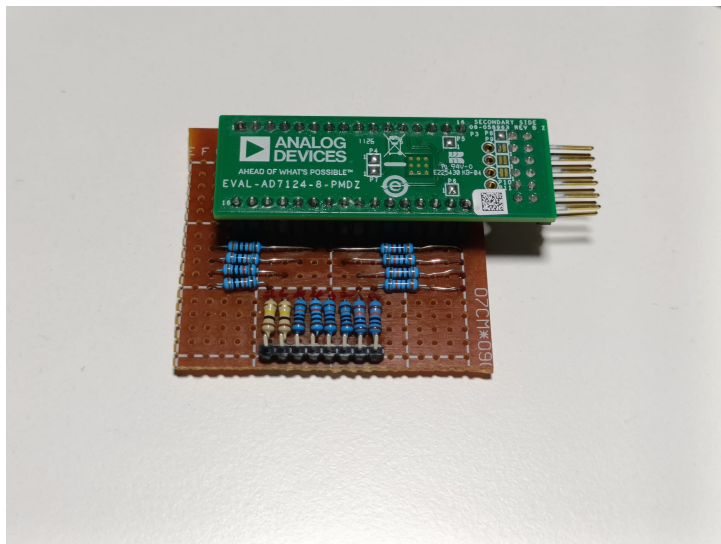


Figura 1.7: Plăcuță cu divizoarele rezistive de atenuare și convertorul AD7124-8.

1.2.5. Fabricarea prin printare 3D

Toate piesele mecanice ale brațului au fost fabricate prin printare 3D, la domiciliu, folosind o imprimantă Bambu Lab A1. Pentru a obține piese cât mai precise și cu o

suprafață cât mai îngrijită, înălțimea unui strat a fost setată la o valoare mică, de 0,08 mm. Această alegere a crescut durata de printare, dar a dus la piese de o calitate foarte bună, cu detalii fine și cu o rezistență corespunzătoare solicitărilor mecanice la care sunt supuse.

Un criteriu important în alegerea modelelor a fost prezența unor canale interioare prin care firele de acționare să poată circula fără obstacole. Toate piesele printate dispun de astfel de canale, vizibile în Figura 1.8, ceea ce permite firelor să treacă prin interiorul degetelor și al palmei în mod ghidat, fără să se agațe sau să se frece de pereți. Această particularitate s-a dovedit esențială pentru buna funcționare a sistemului de acționare prin tendoane.

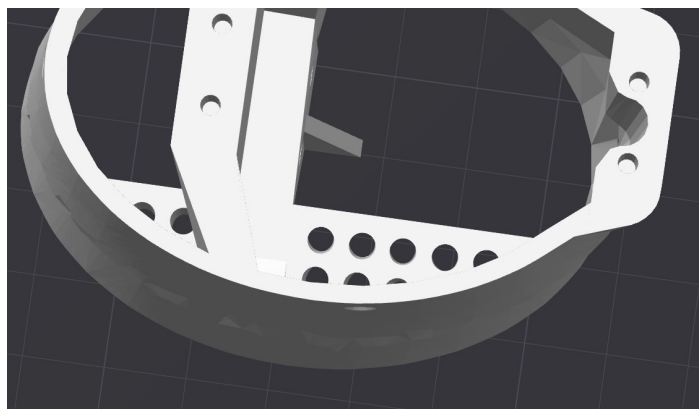


Figura 1.8: Canalele interioare de ghidare a firelor în piesele printate 3D.

În final, după combinarea modelelor, montarea servomotoarelor, a sistemului de acționare și a celui de feedback, a rezultat ansamblul complet prezentat în Figura 1.9. Se observă mâna cu cele cinci degete, palma, încheietura și antebrațul care găzduiește servomotoarele, pulley-urile și fasciculul de fire.

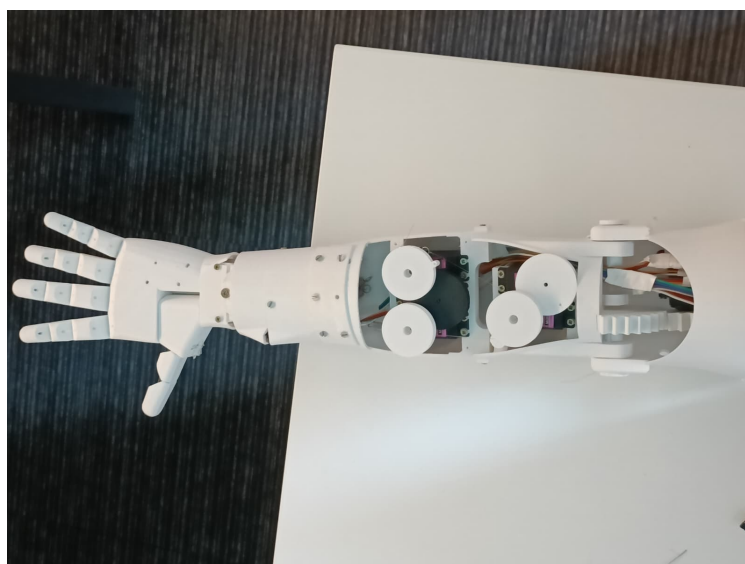


Figura 1.9: Ansamblul complet al brațului robotic.

1.3. Realizarea montajului electronic

Partea electronică a sistemului leagă placa FPGA de servomotoare și de convertorul analog-digital, respectând principiile de izolare și de alimentare fundamentate în capitolul de analiză. Realizarea ei a pornit de la alegerea componentelor potrivite și a continuat cu construirea efectivă a montajului, etapă care a ridicat o serie de provocări practice.

1.3.1. Izolarea și translatarea de nivel a semnalelor PWM

Primul pas a fost identificarea unei componente de izolare potrivite din gama Analog Devices, alegerea oprindu-se asupra izolatorului ADuM1400. Pe lângă rolul de izolare galvanică, acesta rezolvă și o a doua problemă, la fel de importantă. Placa Basys3 scoate semnalele logice la 3,3 V, în timp ce servomotoarele MG996R așteaptă semnal PWM la nivel de 5 V. Izolatorul ADuM1400 dispune de alimentări independente pe cele două laturi, fiind alimentat la 3,3 V pe partea dinspre placă și la 5 V pe partea dinspre servomotoare, astfel încât realizează în același timp și o translatare a nivelului logic de la 3,3 V la 5 V. În felul acesta, o singură componentă rezolvă atât izolarea, cât și adaptarea de nivel necesară comenzii corecte a servomotoarelor.

Pentru a putea lucra cu acest izolator a fost folosită placa de evaluare dedicată, care include o zonă de prototipare deosebit de utilă. Pe această zonă a putut fi gândit dinainte modul de amplasare a componentelor, profitând de traseele de alimentare și de date deja existente pe placă. În cadrul acestei etape a fost identificată și necesitatea înlăturării unei rezistențe de 0 Ω de pe placa de evaluare, care altfel ar fi cerut o alimentare suplimentară fără rost în configurația folosită. Modul de amplasare gândit pe zona de prototipare este prezentat în Figura 1.10, unde sunt marcate canalele pe care urmau să fie lipiți pinii și conectate servomotoarele, atât alimentarea, cât și liniile de date.

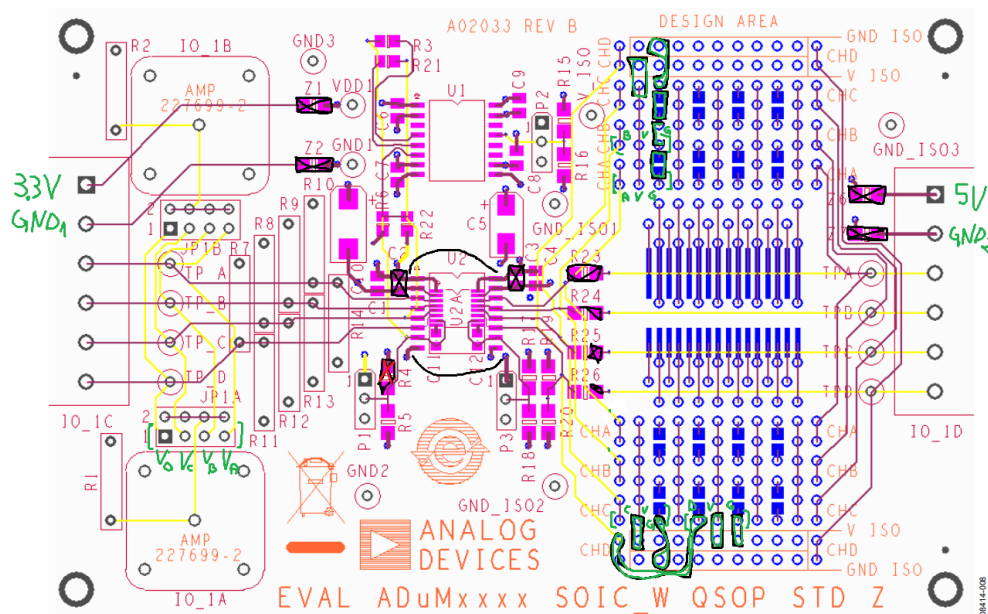


Figura 1.10: Amplasarea componentelor pe zona de prototipare a plăcii de evaluare ADuM1400. Cu verde sunt marcate canalele pentru pinii servomotoarelor.

Întrucât un singur izolator ADuM1400 dispune de patru canale, iar sistemul are nevoie de opt semnale PWM, au fost folosite două astfel de plăci. Izolatoarele au fost

lipite, la fel ca restul componentelor, individual. Între pinii de alimentare ai servomotoarelor a fost adăugat un condensator de aproximativ $70 \mu\text{F}$, pentru a asigura o funcționare stabilă a acestora și a acoperi vârfurile de curent care apar la pornirea motoarelor.

1.3.2. Bucla de feedback și normalizarea semnalelor

Pe calea de feedback, semnalele de poziție provenite de la servomotoare sunt citite de convertorul AD7124-8, folosind intrările sale de la 0 la 7. Comunicația dintre convertor și placa Basys3 trece prin izolatorul SPI MAX14850, conectat la rândul lui la conectorul Pmod al plăcii. Astfel, întregul lanț de feedback rămâne separat galvanic de partea de putere.

Așa cum s-a arătat în secțiunea dedicată sistemului de feedback, semnalele provenite de la potențiometrele servomotoarelor pot ajunge până la aproximativ $2,9 \text{ V}$, ceea ce depășește tensiunea de referință de $2,5 \text{ V}$ a convertorului. Pentru a le aduce sub această limită, fiecare semnal trece printr-un divizor rezistiv, realizat pe o plăcuță separată. Divizorul folosește o rezistență $R_1 = 100 \text{ k}\Omega$ în calea semnalului și una $R_2 = 300 \text{ k}\Omega$ spre masă, conform schemei din Figura 1.11. Tensiunea de ieșire rezultă din raportul divizorului,

$$V_{\text{ies}} = V_{\text{in}} \cdot \frac{R_2}{R_1 + R_2} = V_{\text{in}} \cdot \frac{300}{100 + 300} = 0,75 \cdot V_{\text{in}},$$

astfel încât tensiunea maximă de $2,9 \text{ V}$ devine aproximativ $2,18 \text{ V}$, încadrându-se confortabil sub referința convertorului. Montajul efectiv al acestor divizoare, împreună cu placa convertorului, este prezentat în Figura 1.7.

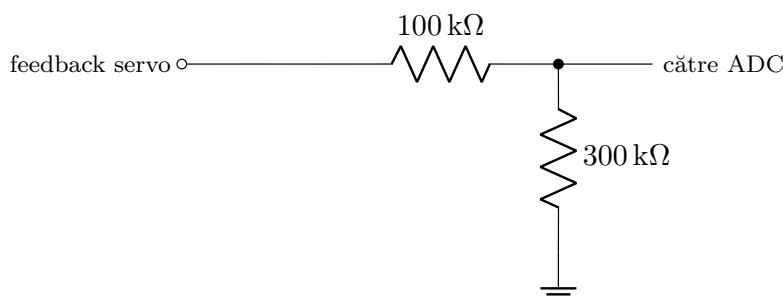


Figura 1.11: Schema unui divizor rezistiv de pe calea de feedback.

1.3.3. Alimentarea montajului

Alimentarea întregului montaj pornește de la o sursă externă și este distribuită prin modulul DC2468A, care furnizează tensiunea de 5 V necesară servomotoarelor. Tot din acesta este alimentat și regulatorul liniar ADP165, care coboară tensiunea la $3,3 \text{ V}$ pentru convertorul analog-digital, asigurându-i o alimentare curată, separată de cea a servomotoarelor. Lanțul de alimentare urmează astfel un traseu clar, de la sursă către DC2468A, iar de acolo, pe de o parte direct la servomotoare, iar pe de altă parte prin ADP165 către convertor.

Modulul DC2468A permite și modificarea unei rezistențe pentru a obține o tensiune de 6 V , care ar fi corespuns funcționării la putere maximă a servomotoarelor. Această modificare nu a fost însă necesară în configurația actuală și rămâne o posibilă direcție de dezvoltare ulterioară. Întregul ansamblu electronic, cu toate plăcile interconectate, este prezentat în Figura 1.12.

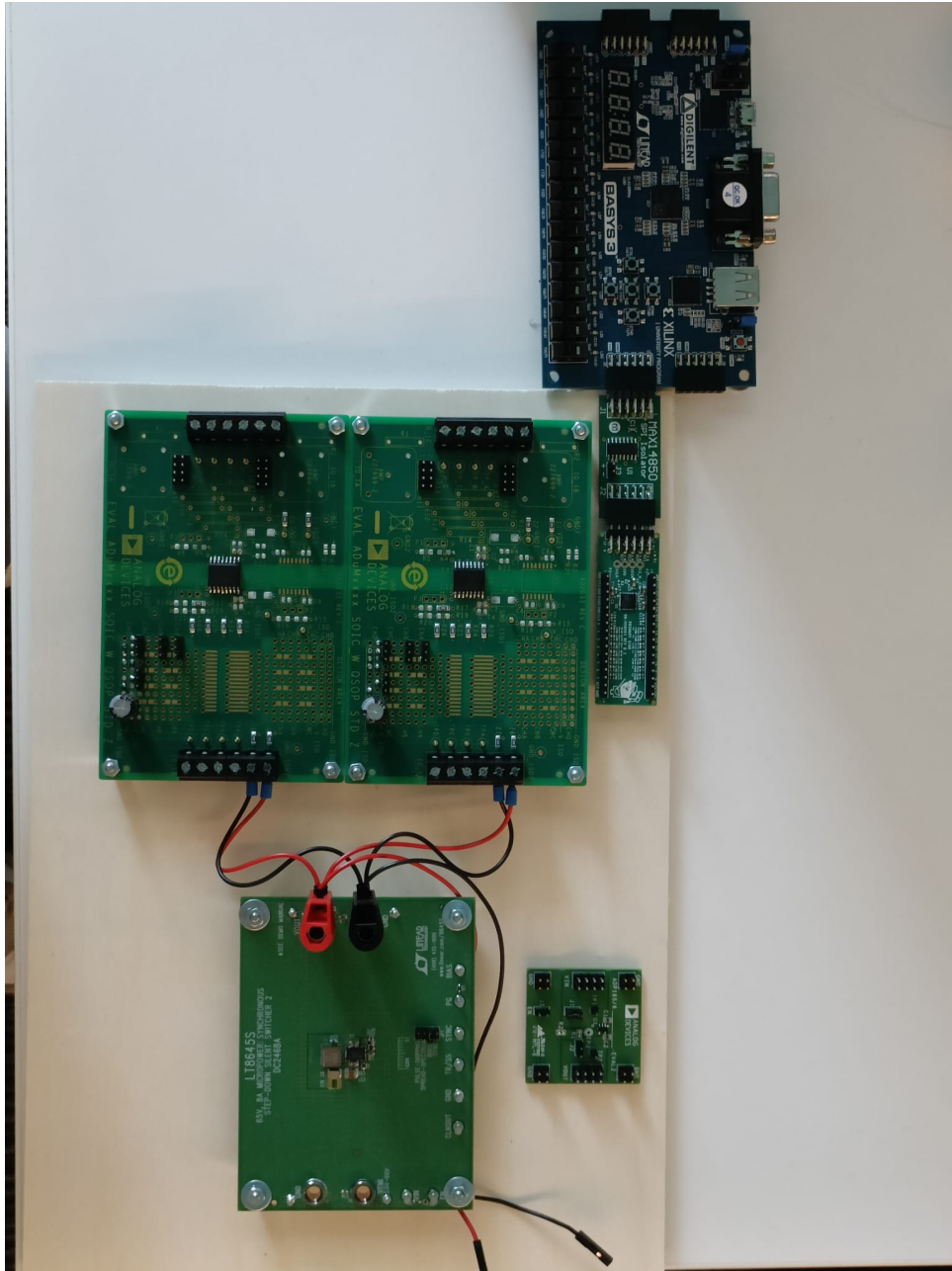


Figura 1.12: Ansamblul electronic complet, cu placa Basys3, cele două izolatoare ADuM1400, izolatorul SPI MAX14850, convertorul AD7124-8 și sursele de alimentare.

1.4. Implementarea pe placa FPGA

Partea digitală a sistemului reprezintă contribuția centrală a acestei lucrări și este descrisă în VHDL ca un ansamblu de module care funcționează în paralel, sincronizate de același ceas de sistem. Această secțiune prezintă arhitectura internă a circuitului și detaliază, pe rând, modulele care o compun, insistând asupra celor care au ridicat cele mai multe probleme de proiectare.

1.4.1. Arhitectura modulară

Întregul circuit este organizat în jurul unui modul de nivel superior, care instanțiază și interconectează modulele specializate. Organizarea acestora și fluxul datelor între ele sunt prezentate în Figura 1.13.

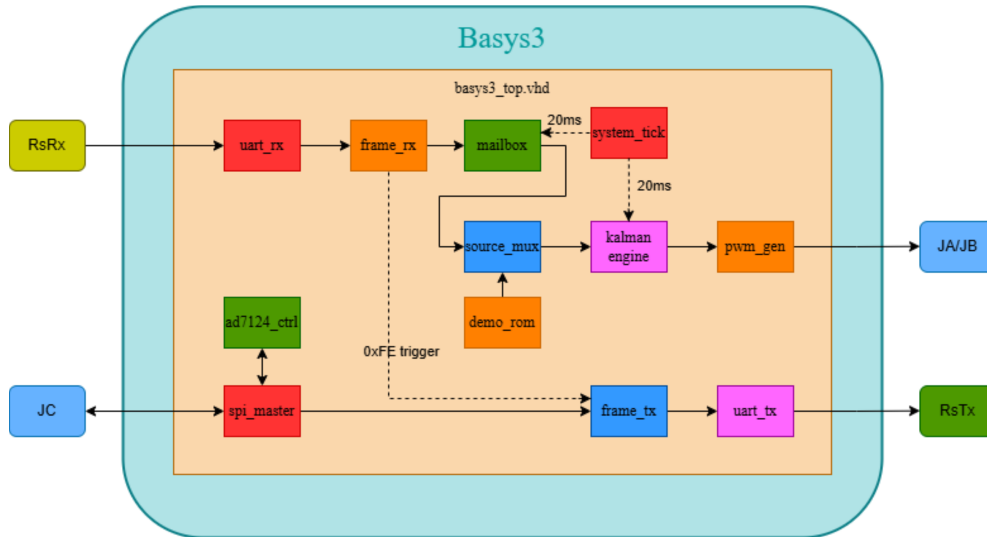


Figura 1.13: Arhitectura internă a circuitului implementat pe placa Basys3, cu modulele componente și fluxul datelor între acestea.

Pe calea principală de comandă, datele intră prin modulul `uart_rx`, care recuperează octeții de pe linia serială, sunt asamblate în cadre de către `frame_rx` și depuse în `mailbox`. De acolo, printr-un bloc de selecție a sursei, ajung la `kalman_engine`, care le filtrează, iar rezultatul este transformat în semnale de comandă de către generatoarele `pwm_gen`. Pe calea de feedback, modulul `ad7124_ctrl` comandă convertorul prin `spi_master` și recuperează pozițiile măsurate. Un modul `system_tick_gen` produce semnalul de sincronizare de 20 ms care ritmează întregul sistem, iar modulele `demo_rom` și `frame_tx` asigură modul demonstrativ, respectiv transmiterea valorilor înapoi către calculator.

1.4.2. Generarea semnalului de sincronizare

Ritmul întregului sistem este dat de un semnal care marchează începutul fiecărui ciclu de control de 20 ms. Acesta este produs de un numărător care pornește de la ceasul de 100 MHz și generează un impuls scurt la fiecare 2.000.000 de perioade de ceas, ceea ce corespunde exact intervalului de 20 ms. Toate celelalte module se raportează la acest semnal, astfel încât filtrarea, citirea convertorului și actualizarea comenzilor se desfășoară sincronizat.

1.4.3. Recepția și validarea cadrelor

Recepția datelor se realizează în doi pași. Mai întâi, modulul `uart_rx` reconstituie fiecare octet de pe linia serială, eșantionând biții la mijlocul intervalului lor pentru a maximiza marja față de tranziții. Pentru a evita problemele de metastabilitate, linia de intrare este trecută mai întâi prin două bistabile înseriate, înainte de a fi folosită.

Octeții astfel recuperați sunt preluați de modulul `frame_rx`, care îi assemblează într-un cadru complet și îi validează. Acesta funcționează ca o mașină cu stări, care

așteaptă mai întâi octetul de direcție, apoi colectează cele opt unghiuri și, în final, verifică suma de control. Verificarea se bazează pe proprietatea operației SAU exclusiv, conform căreia suma de control a unui cadru corect, combinată din nou cu toți octeții, trebuie să dea zero.

```

1 when S_RECV_CHKSUM =>
2   if rx_valid = '1' then
3     if (xor_acc xor rx_data) = x"00" then
4       frame_valid  <= '1';
5       frame_dir    <= dir_reg;
6       frame_angles <= angles_reg;
7     else
8       checksum_err <= '1';
9     end if;
10    state <= S_WAIT_DIR;
11  end if;

```

Listing 1.1: Verificarea sumei de control la recepție.

Pentru a nu rămâne blocat pe un cadru incomplet, modulul dispune și de un mecanism de expirare, care readuce mașina de stări în starea inițială dacă un cadru nu este finalizat într-un interval rezonabil.

1.4.4. Sincronizarea prin mailbox

Unul dintre cele mai importante module pentru funcționarea corectă a sistemului este **mailbox**, care rezolvă problema sincronizării dintre recepție și prelucrare. Cadrele sosesc prin UART la momente independente de ciclul intern de 20 ms, astfel încât o citire directă a datelor de către filtru ar putea surprinde un set de unghiuri pe jumătate actualizat. Mecanismul folosește două registre. Datele nou primite sunt scrise într-un registru de intrare și marcate printr-un fanion, iar copierea lor în registrul din care citește filtrul se face dintr-o singură mișcare, doar la începutul ciclului de control.

```

1 if wr_valid = '1' then
2   buf_write  <= wr_angles;
3   new_pending <= '1';
4 end if;
5
6 if tick_20ms = '1' then
7   if new_pending = '1' then
8     buf_read  <= buf_write;
9     new_pending <= '0';
10    rd_new    <= '1';
11  end if;
12 end if;

```

Listing 1.2: Comutarea datelor în mailbox, sincronizată cu ciclul de 20 ms.

În acest fel, filtrul lucrează întotdeauna pe un set de date coerent, în timp ce recepția poate continua nestingherită în fundal.

1.4.5. Implementarea filtrului Kalman

Filtrul Kalman constituie nucleul prelucrării și este, totodată, modulul cel mai complex din întregul circuit. Modelul matematic, cu cele două stări, poziția și viteza, a fost dedus în capitolul de analiză. Aici este prezentat modul în care ecuațiile aceluia model au fost transpuse în hardware, cu toate adaptările impuse de lucrul în aritmetică

cu virgulă fixă și de constrângerile de timp ale plăcii. Întregul filtru este implementat ca o mașină cu stări care parcurge, pentru fiecare dintre cele opt canale, etapele de încărcare a stării, predicție, actualizare și stocare a rezultatului, conform diagramei din Figura 1.14.

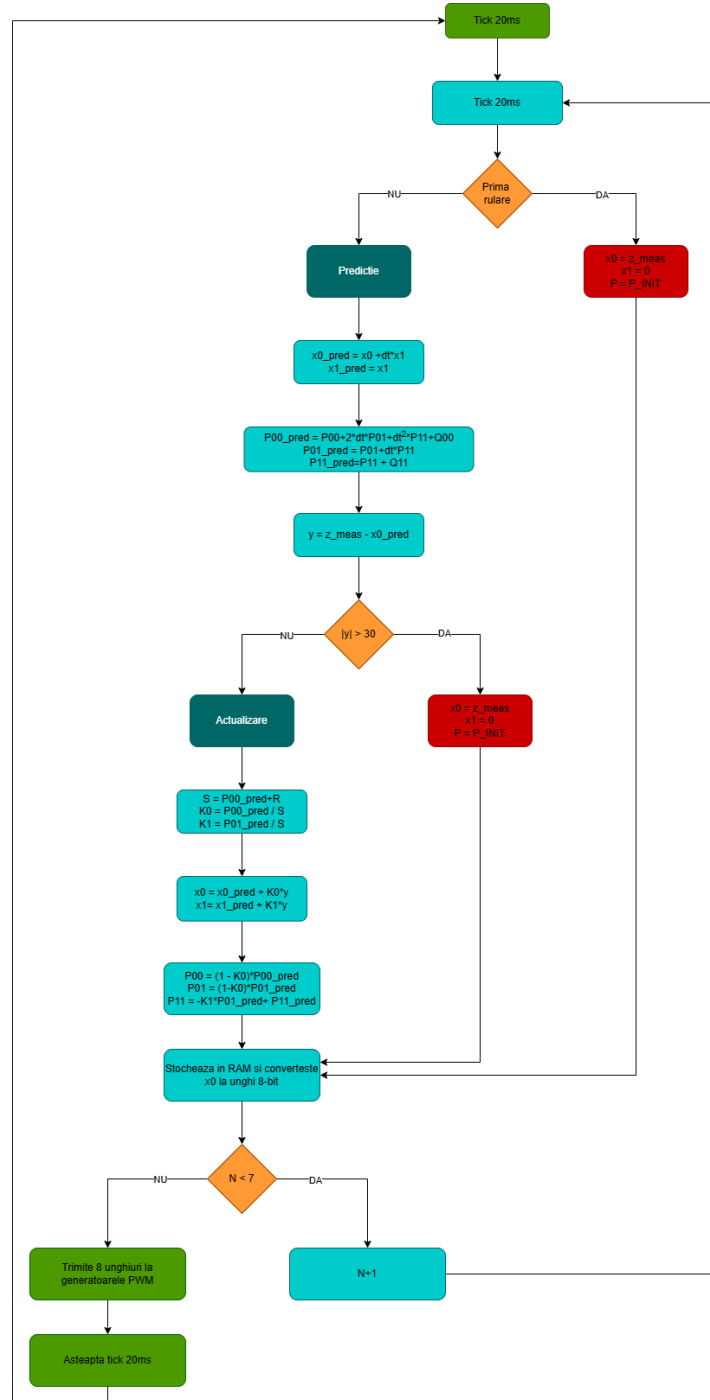


Figura 1.14: Mașina cu stări a filtrului Kalman, parcursă pentru fiecare canal.

Reprezentarea în virgulă fixă

Toate mărimile filtrului sunt reprezentate în formatul Q16.16 prezentat în capitolul de analiză, adică prin întregi pe 32 de biți obținuți prin înmulțirea valorii reale cu 2^{16} . Constantele filtrului au fost calculate o singură dată și înscrise direct în cod sub această

formă. Astfel, intervalul de eșantionare $\Delta t = 0,02$ s devine $0,02 \times 65536 \approx 1311$, zgomotul de proces pe poziție $Q_{00} = 0,01$ devine 655, zgomotul pe viteză $Q_{11} = 0,1$ devine 6554, iar incertitudinea inițială $P_{\text{init}} = 10$ devine 655360. Aceste valori se regăsesc ca atare printre constantele modului.

```

1 constant DT_Q16   : signed(31 downto 0) := to_signed(1311, 32);
2 constant Q00      : signed(31 downto 0) := to_signed(655, 32);
3 constant Q11      : signed(31 downto 0) := to_signed(6554, 32);
4 constant R_NOISE   : signed(31 downto 0) := to_signed(65536, 32);
5 constant P_INIT    : signed(31 downto 0) := to_signed(655360, 32);

```

Listing 1.3: Constantele filtrului, în format Q16.16.

Înmulțirea a două numere în acest format produce un rezultat scalat cu 2^{32} , motiv pentru care produsul pe 64 de biți trebuie deplasat înapoi cu 16 biți, prin extragerea biților de la 16 la 47. Această operație este încapsulată într-o funcție folosită în tot restul modului.

```

1 function mul_q16(a : signed(31 downto 0); b : signed(31 downto 0))
2     return signed is
3     variable product : signed(63 downto 0);
4 begin
5     product := a * b;
6     return product(47 downto 16);
7 end function;

```

Listing 1.4: Funcția de înmulțire în virgulă fixă Q16.16.

Etapa de predicție

În etapa de predicție, poziția este avansată cu termenul corespunzător vitezei, iar matricea de covarianță este propagată conform ecuațiilor deduse în capitolul de analiză. Predicția covarianței,

$$\begin{aligned}
 P_{00}^- &= P_{00} + 2\Delta t P_{01} + \Delta t^2 P_{11} + Q_{00}, \\
 P_{01}^- &= P_{01} + \Delta t P_{11}, \\
 P_{11}^- &= P_{11} + Q_{11},
 \end{aligned}$$

nu poate fi calculată dintr-o singură expresie, deoarece ar conține mai multe înmulțiri în lanțuite, ceea ce ar crea o cale combinațională prea lungă. De aceea, ea este descompusă pe mai multe stări, fiecare efectuând cel mult o înmulțire. Mai întâi se pre-calculează produsele $\Delta t \cdot P_{01}$ și $\Delta t \cdot P_{11}$, iar apoi acestea sunt combinate prin adunări succesive, după cum se vede în fragmentul de mai jos, unde `dt_p01` și `dt_p11` sunt produsele pre-calculate.

```

1 when S_PREDICT_DT =>
2     dt_p01 <= mul_q16(DT_Q16, p01);
3     dt_p11 <= mul_q16(DT_Q16, p11);
4     fsm    <= S_PREDICT_P00A;
5
6 when S_PREDICT_P00A =>
7     pp00 <= p00 + dt_p01 + dt_p01;    -- p00 + 2*dt*p01
8     pp01 <= p01 + dt_p11;            -- p01 + dt*p11
9     fsm  <= S_PREDICT_P00B;

```

Listing 1.5: Predicția covarianței, descompusă pe stări.

Etapa de actualizare

În etapa de actualizare se calculează mai întâi inovația, adică diferența dintre poziția măsurată și cea prezisă, $y = z - \hat{x}_0^-$, și covarianța inovației $S = P_{00}^- + R$. Câștigul Kalman se obține apoi prin cele două împărțiri $K_0 = P_{00}^-/S$ și $K_1 = P_{01}^-/S$, realizate cu modulul de împărțire descris în continuare. Starea este în final corectată proporțional cu inovația,

$$\hat{x}_0 = \hat{x}_0^- + K_0 y, \quad \hat{x}_1 = \hat{x}_1^- + K_1 y,$$

fiecare dintre aceste corecții fiind descompusă, la rândul ei, într-un pas de înmulțire și unul de adunare, pentru a respecta disciplina unei singure înmulțiri pe ciclu de ceas.

```

1 when S_UPDATE_X0A =>
2     mul_res <= mul_q16(k0, y_innov);    -- K0 * y
3     fsm    <= S_UPDATE_X0B;
4
5 when S_UPDATE_X0B =>
6     x0 <= x0_pred + mul_res;            -- x0_pred + K0*y
7     fsm  <= S_CLAMP_X0;

```

Listing 1.6: Corecția poziției, separată în înmulțire și adunare.

Stocarea stării și mecanisme de protecție

O decizie de proiectare importantă privește modul de stocare a stării celor opt canale. Fiecare canal are nevoie de cinci valori, poziția, viteza și cele trei elemente ale matricei de covarianță. În loc de a folosi o singură memorie comună, care ar fi impus un multiplexor mare și greu de încadrat în timp, starea a fost distribuită în cinci memorii separate, câte una pentru fiecare mărime, fiecare cu opt locații. Această separare simplifică accesul și ușurează respectarea constrângerilor de timp, aspect detaliat în secțiunea dedicată închiderii temporale.

Filtrul conține și un mecanism de protecție împotriva valorilor aberante. Atunci când inovația depășește în modul un prag stabilit, echivalentul a 30 de grade, filtrul abandonează corecția obișnuită și reinițializează starea canalului respectiv, considerând că o asemenea diferență provine dintr-o citire eronată și nu dintr-o mișcare reală. De asemenea, poziția estimată este limitată la intervalul de 0 până la 180 de grade, iar viteza la o valoare maximă rezonabilă, pentru a preveni acumularea unor valori nefirești în urma unor perturbații.

1.4.6. Împărțitorul în virgulă fixă

Calculul câștigului Kalman presupune două împărțiri pentru fiecare canal, iar împărțirea este operația cea mai costisitoare din întregul algoritm. O realizare directă a ei ar fi creat o cale combinațională mult prea lungă pentru a se încadra într-o perioadă de ceas de 10 ns. Din acest motiv, împărțirea a fost implementată într-un modul separat, **divider**, care folosește un algoritm secvențial cu refacere, producând câte un bit de rezultat pe ciclu de ceas. Operația se desfășoară astfel pe parcursul mai multor cicluri, fiecare cu o cale combinațională scurtă, ceea ce permite circuitului să funcționeze la frecvența dorită. Filtrul Kalman pornește împărțirea și așteaptă semnalul de finalizare înainte de a folosi rezultatul.

1.4.7. Generarea semnalelor PWM

Pentru fiecare dintre cele opt canale, un modul **pwm_gen** transformă unghiul filtrat într-un semnal cu durata impulsului corespunzătoare. Modulul folosește un numărător care parcurge ciclic valorile de la 0 la 1.999.999, corespunzând perioadei de 20 ms, și menține ieșirea la nivel ridicat atâta timp cât numărătorul este sub un prag calculat din unghi.

Deși specificația nominală a servomotoarelor prevede un impuls între 1 și 2 ms, testele practice au arătat că servomotoarele MG996R folosite nu parcurg întreaga cursă la acest interval, oprindu-se mult înainte de a atinge extremele. Din acest motiv, intervalul a fost lărgit la valorile folosite și de bibliotecile de control uzuale pentru astfel de servomotoare, de la 0,544 ms pentru 0 grade până la 2,4 ms pentru 180 de grade. La frecvența de 100 MHz, impulsul minim corespunde la 54400 de perioade de ceas, iar fiecare grad adaugă încă $185600/180 \approx 1031$ de perioade, astfel încât pragul se calculează ca

$$\text{prag}(\alpha) = 54400 + \alpha \times 1031,$$

relație transpusă direct în cod prin expresia de mai jos.

```
1 threshold <= to_unsigned(54400, 21) +
2   resize(unsigned(angle) * to_unsigned(1031, 11), 21);
```

Listing 1.7: Calculul pragului de comparație din unghi.

Cele opt instanțe funcționează independent și în paralel, astfel încât actualizarea unui canal nu le afectează pe celelalte.

Un caz aparte îl constituie canalul corespunzător mișcării palmare a degetului mare. Acest servomotor este montat în sens invers față de celelalte, din cauza constrângerilor mecanice de amplasare, și are nevoie de o cursă redusă, între 90 și 0 grade, în care valoarea mică deschide articulația, iar cea mare o închide. Pentru a respecta acest comportament fără a modifica restul logicii, comanda acestui canal este remapată separat înainte de a ajunge la generatorul PWM. Unghiul primit este redus la jumătate și scăzut din 90, astfel încât o comandă de 0 grade produce o ieșire de 90 de grade, iar una de 180 de grade produce o ieșire de 0 grade.

```

1 pwm_angles(31 downto 24) <=
2   std_logic_vector(to_unsigned(90, 8) -
3     shift_right(unsigned(kalman_out(31 downto 24)), 1)
4   );

```

Listing 1.8: Remaparea canalului degetului mare, inversat și scalat.

1.4.8. Interfața cu convertorul analog-digital

Citirea poziției reale a servomotoarelor este gestionată de modulul `ad7124_ctrl`, în colaborare cu modulul generic `spi_master`. La pornire, controlerul parcurge o secvență de inițializare prin care configurează registrele convertorului, stabilind modul de conversie continuă, filtrul digital și ordinea canalelor. Ulterior, citește ciclic valorile măsurate, folosind un mod în care fiecare conversie este însoțită de identificatorul canalului căruia îi aparține, ceea ce simplifică asocierea măsurărilor cu articulațiile. Valoarea brută pe 24 de biți este apoi convertită într-un unghi, prin scalarea la intervalul de 0 până la 180 de grade.

Modulul `spi_master` implementează protocolul SPI în modul cerut de convertor și derivă frecvența magistralei din ceasul de sistem, obținând cei 5 MHz folosiți pentru comunicație. La fel ca la recepția UART, linia de intrare dinspre convertor este trecută prin două bistabile înseriate, pentru a preveni metastabilitatea la trecerea între domeniile de timp.

1.4.9. Modurile de funcționare

Pe lângă funcționarea comandată de la calculator, sistemul mai dispune de două moduri de lucru, selectabile prin comutatoarele de pe placă, ceea ce permite folosirea lui și în absența conexiunii cu calculatorul gazdă. Sursa unghiurilor trimise către filtru este aleasă în funcție de starea comutatoarelor.

În modul normal, unghiurile provin din cutia poștală, adică de la cadrele primite prin UART. Un prim comutator activează modul demonstrativ, în care sursa devine modulul `demo_rom`, o memorie care conține o secvență predefinită de mișcări, parcursă pas cu pas la fiecare ciclu de control. În acest mod, mâna execută o demonstrație repetitivă, utilă pentru testarea mecanică a ansamblului și pentru prezentarea funcționării independent de partea de viziune.

Un al doilea comutator activează un mod manual, în care fiecare dintre cele opt articulații este comandată individual de câte un comutator dedicat. Un comutator ridicat trimite articulația corespunzătoare către poziția de 180 de grade, iar unul coborât către 0 grade. Comanda astfel obținută trece prin aceleași etape de filtrare și de generare a semnalelor PWM ca în celelalte moduri, astfel încât mișcarea rămâne lină. Acest mod s-a dovedit deosebit de util pentru verificarea individuală a fiecărui servomotor și pentru reglarea mecanică a firelor și a tensiunii arcurilor.

1.5. Închiderea temporală a circuitului

Așa cum s-a arătat în capitolul de analiză, provocarea reală a acestui sistem nu este debitul de calcul, ci încadrarea fiecărei operații într-o singură perioadă de ceas de 10 ns, astfel încât circuitul să funcționeze stabil la 100 MHz. Această secțiune descrie problemele de închidere temporală întâmpinate și soluțiile aplicate pentru a le rezolva.

1.5.1. Problema inițială

În primele variante ale implementării, filtrul Kalman conținea expresii aritmetice complexe evaluate într-un singur pas, precum calculul complet al unei componente a matricei de covarianță sau corecția stării cu un termen proporțional cu inovația. Fiecare astfel de expresie înlanțuia mai multe înmulțiri și adunări, formând o cale combinațională a cărei întârziere depășea cu mult perioada de 10 ns. La sinteză și implementare, unealta raporta un slack negativ, adică circuitul nu putea funcționa la frecvența dorită, întârzierea celui mai lung traseu depășind bugetul disponibil între două fronturi de ceas.

O a doua sursă de probleme a fost modul inițial de stocare a stării filtrului. Toate mărimile celor opt canale fuseseră grupate într-o singură memorie, ceea ce impunea, la fiecare acces, un multiplexor foarte larg pentru a selecta canalul dorit. Acest multiplexor adăuga o întârziere suplimentară importantă pe calea critică, agravând problema de timing.

1.5.2. Soluțiile aplicate

Prima măsură a fost impunerea unei discipline stricte în mașina cu stări a filtrului, aceea de a efectua cel mult o singură înmulțire pe ciclu de ceas. Toate expresiile compuse au fost descompuse în pași elementari, fiecare cu o cale combinațională scurtă, așa cum s-a arătat în secțiunea dedicată filtrului. Această descompunere a crescut numărul de stări parcurse, dar a scurtat drastic calea critică, în schimbul câtorva cicluri de ceas suplimentare, complet neglijabile față de bugetul de 20 ms.

A doua măsură a vizat operația de împărțire, cea mai costisitoare din algoritm. Realizarea ei directă ar fi produs o cale combinațională mult prea lungă, motiv pentru care a fost implementată ca un modul secvențial separat, care produce câte un bit de rezultat pe ciclu de ceas. În acest fel, întârzierea împărțirii este distribuită pe mai multe cicluri, fiecare respectând constrângerea de 10 ns.

A treia măsură a fost separarea stării filtrului în cinci memorii distincte, câte una pentru fiecare mărime, în locul unei singure memorii comune. Această separare a eliminat multiplexorul larg de pe calea critică și a permis, în plus, ca memoriile să fie realizate ca memorie distribuită în celulele logice, fără a ocupa blocuri de memorie dedicate.

1.5.3. Rezultatele obținute

După aplicarea acestor soluții, circuitul respectă toate constrângerile temporale la frecvența de 100 MHz. Închiderea temporală s-a obținut totuși la limită, ceea ce arată că soluțiile au fost necesare. În fazele intermediare ale implementării, după plasare și rutare, unealta raporta încă un slack negativ, de ordinul a $-1,8$ ns, iar abia etapa finală de optimizare fizică a reușit să aducă slack-ul la o valoare pozitivă. Valorile finale ale analizei temporale sunt prezentate în Tabelul 1.1.

Tabela 1.1: Rezultatele finale ale analizei temporale la 100 MHz.

Parametru	Valoare
Perioada de ceas impusă	10,000 ns
Worst Negative Slack (WNS)	0,002 ns
Total Negative Slack (TNS)	0,000 ns
Worst Hold Slack (WHS)	0,038 ns
Puncte terminale în eșec	0

Un slack pozitiv, chiar și foarte mic, înseamnă că circuitul funcționează corect la frecvența cerută, iar toate cele peste cinci mii de puncte terminale ale proiectului respectă constrângerile, fără nicio cale în eșec.

1.5.4. Utilizarea resurselor

Dincolo de închiderea temporală, implementarea se remarcă printr-un consum foarte redus de resurse în raport cu cele disponibile pe circuitul Artix-7 XC7A35T. Valorile raportate de unealtă sunt sintetizate în Tabelul 1.2.

Tabela 1.2: Utilizarea resurselor pe circuitul Artix-7 XC7A35T.

Resursă	Utilizat	Disponibil	Procent
Tabele de căutare (LUT)	3226	20800	15,5%
Bistabile (FF)	1730	41600	4,2%
Blocuri de memorie (BRAM)	0	50	0%
Blocuri aritmetice (DSP)	28	90	31,1%

Se remarcă în mod deosebit faptul că proiectul nu folosește niciun bloc de memorie dedicat, cele cinci memorii ale filtrului fiind realizate integral ca memorie distribuită în celulele logice, urmare directă a separării descrise mai sus. Consumul cel mai ridicat, de aproximativ o treime, apare la blocurile aritmetice dedicate, folosite pentru înmulțirile din filtrul Kalman și din generarea semnalelor PWM. Marja largă rămasă la toate categoriile de resurse confirmă că platforma aleasă are capacitatea necesară nu doar pentru sistemul actual, ci și pentru eventuale extinderi viitoare.

Bibliografie